PBCST304

OBJECT ORIENTED PROGRAMMING

MODULE - 1

Prepared by: Aswathy.T.K

Introduction to Java:

Structure of a simple java program; Java programming Environment and Runtime Environment (Command Line & IDE); Java compiler; Java Virtual Machine; Primitive Data types and Wrapper Types; Casting and Autoboxing; Arrays; Strings; Vector class; Operators - Arithmetic, Bitwise, Relational, Boolean Logical, Assignment, Conditional (Ternary); Operator Precedence; Control Statements - Selection Statements, Iteration Statements and Jump Statements; Functions; Command Line Arguments; Variable Length Arguments; Classes; Abstract Classes; Interfaces. [***Use proper naming conventions***]

OOP Concepts :-

Data abstraction, encapsulation, inheritance, polymorphism, Procedural and object oriented programming paradigm; Microservices.

Object Oriented Programming in Java :-

Declaring Objects; Object Reference; Introduction to Methods; Constructors; Access Modifiers; ***this*** keyword.

INTRODUCTION TO JAVA

- Java is a **high-level, object-oriented, platform-independent** programming language developed by **James Gosling** and his team at Sun Microsystems in 1991.
- The language was initially called ‘Oak’
- 1995: Oak was renamed Java and officially released to the public.
- It quickly became popular for developing web applications.
- Today, Java is maintained by Oracle Corporation.

James Gosling



KEY FEATURES OF JAVA??

- S – Simple
- O – Object-Oriented
- P – Platform Independent
- S – Secure
- R – Robust
- P – Portable
- M – Multithreaded
- A – Architecture Neutral
- H – High Performance
- D – Distributed
- D – Dynamic

Buzzwords in Java

Mnemonic to Remember the Features:

*"**S**ome **O**ld **P**rogrammers **S**ay **R**obots **P**refer **M**aking **A**pplications **H**ighly **D**ynamic."*

1. Simple:

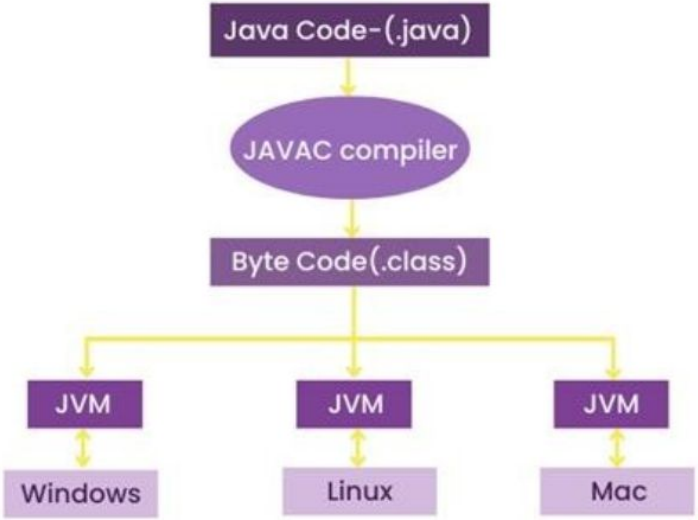
- Java is simple because it was designed to be **easier to learn and use** than earlier programming languages like C++.
- Easy Syntax.
- No Pointers.
- Automatic Memory Management.
  - Java has a **Garbage Collector** that automatically removes unused objects from memory.
  - Programmers do not need to manually allocate and free memory.
- Fewer complex features.
- Rich Standard Library.
  - Java provides a large collection of built-in classes and methods.
  - This reduces the amount of code programmers need to write.

2. Object oriented:

- Java is an object-oriented programming language because **it organizes programs using objects and classes** rather than just functions and procedures.
- Uses Classes and Objects
  - Almost everything is represented as an object, and programs are designed by creating and interacting with these objects.
- Supports the Four Pillars of Object-Oriented Programming (OOP)
  - Encapsulation
  - Inheritance
  - Polymorphism
  - Abstraction

3. Platform Independent:

- A platform-independent language is one whose **programs can run on different operating systems** (such as Windows, Linux, and macOS) **without modifying the source code.**
- Java is platform-independent because of the **Java Virtual Machine (JVM).**



- Java compiler (javac) converts the source code into bytecode.
- The JVM on each operating system interprets or compiles the same bytecode into machine code for that platform.
- Therefore, the same .class file can run on any system that has a compatible JVM installed.
- Since a JVM is available for different operating systems, the same Java program can run on any platform without modification=>known as "**Write Once, Run Anywhere (WORA)**."

- Key Advantage: No need to rewrite code for different operating systems.

4. Secure:

- Java is considered secure because it includes **several built-in security features** that protect programs from common threats.

Why is Java Secure?

- No Explicit Pointers
- Bytecode Verification
- Automatic Garbage Collection
- Strong Type Checking
- Exception Handling

5. Robust:

- Robustness refers to the **ability** of a programming language **to handle errors, invalid inputs, and unexpected situations without crashing or producing incorrect results.**
- Java is considered a robust programming language because it provides:
  - Strong type checking
  - Exception handling
  - Automatic garbage collection
  - No explicit pointers (reduces memory-related errors)
  - Built-in security features

6.Portable:

- Java is portable because a **Java program can run on different operating systems and hardware platforms without modifying its source code.**
- Portable = "Write Once, Run Anywhere (WORA)."
- Suppose we write a Java program on Windows.
- After compiling it, we get a .class file.
- We can copy this file to a Linux or macOS computer that has a JVM installed, and it will run without recompiling.
- ie, Java is portable because its compiled bytecode can run on any platform that has a Java Virtual Machine (JVM)

6. Multithreaded:

- Java is multithreaded because it **allows multiple threads** to run concurrently within a single program.
- This enables a program to perform multiple tasks simultaneously, **improving performance and responsiveness.**

7. Architecture Neutral:

- Java is architecture neutral because its compiled code **(bytecode) is not dependent on any specific processor architecture or hardware**.
- The same bytecode can run on different CPU architectures through the Java Virtual Machine (JVM).

8.High Performance:

- Java is considered high performance because **its bytecode is executed efficiently by the Java Virtual Machine** (JVM), which uses technologies such as the **Just-In-Time (JIT) compiler** to improve execution speed.
- **Just-In-Time (JIT) Compiler**:
  - The JVM includes a JIT compiler that converts frequently used bytecode into native machine code.
  - This machine code runs faster than interpreting bytecode every time.

9.Dynamic & Distributed:

- Java is distributed because it **provides built-in support for developing applications that communicate and share data over a network.**
- Java is dynamic because it can **load classes, libraries, and methods at runtime** (during program execution) instead of requiring everything to be fixed at compile time.

JAVA PROGRAMMING ENVIRONMENT

- The Java Programming Environment refers to the complete setup that allows **developers to write, compile, execute, debug, and develop Java applications.**
- It includes the tools, components, and processes required to develop Java applications.
- Components of the Java Programming Environment;

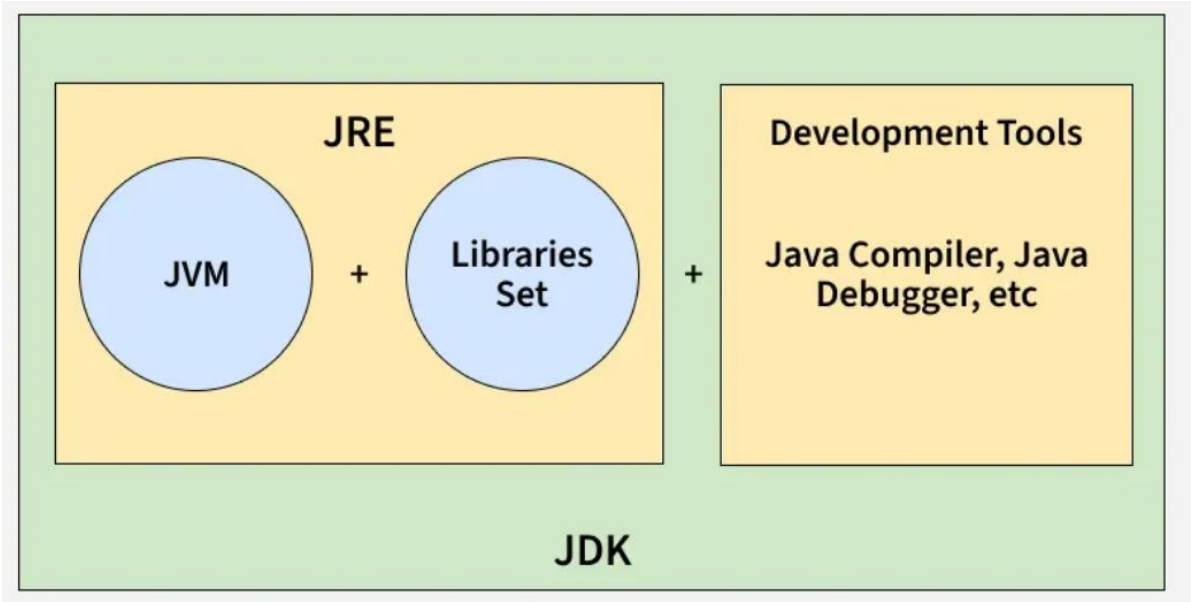
1. **Java Runtime Environment (JRE)**
2. **Java Development Kit (JDK)**
3. **Java Virtual Machine (JVM)**
4. **Java Compiler (javac)**
5. **Debugger**
6. **Documentation tools**
7. **Java libraries**
8. **IDE**

JAVA Runtime Environment (JRE)

- The Java Runtime Environment (JRE) **provides everything needed to execute Java applications.**
- It contains;
  - **Java Virtual Machine (JVM)**
  - **Core class libraries**
  - **Supporting files needed to execute Java bytecode.**
- JRE is a part of the Java Development Kit (JDK) and is responsible for running Java programs on different platforms.
- It does not contain development tools like javac.
- JRE is only used by END-USERS of the system who only wants to run the Java programs.

Note:

- *Java Programming Environment = Develop + Compile + Run*
- *JRE = Run Only*



JAVA Runtime Environment (JRE)

- Command Line
- IDE (Integrated Development Environment)

Command Line:

- Using Java from the Command Line
  - The command line provides a lightweight way to compile and run.
- Prerequisites
  - **Java Development Kit (JDK):**
    - Ensure JDK is installed (e.g., OpenJDK or Oracle JDK).
    - Verify installation by running:

      java -version

      javac -version
  - **A text editor** (e.g., Notepad, VS Code, or Vim) to write Java code.

Steps to Compile and Run a Java Program: Using Command Line

- Write the Java Code:
  - Create a file named HelloWorld.java and type Java program to print Hello World.
- Compile the Code:
  - Open a terminal(command line), navigate to the directory containing HelloWorld.java, and compile using:

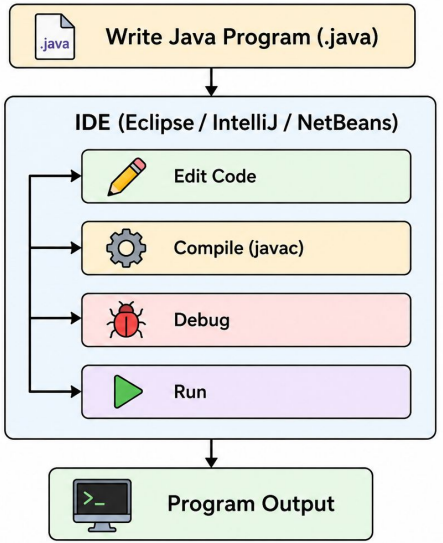
```
javac HelloWorld.java
```

- This generates a HelloWorld.class file if compilation is successful.
- Run the Program:
  - Execute the compiled program using:

```
java HelloWorld
```

Integrated Development Environment (IDE):

- IDE is a software application that provides all the tools needed to write, compile, debug, run, and manage programs in one place.
- Instead of using separate tools for editing, compiling, and debugging, an IDE combines them into a single user-friendly interface.
- Popular Java IDEs;
  - Eclipse IDE
  - IntelliJ IDEA
  - Apache NetBeans
  - Visual Studio Code

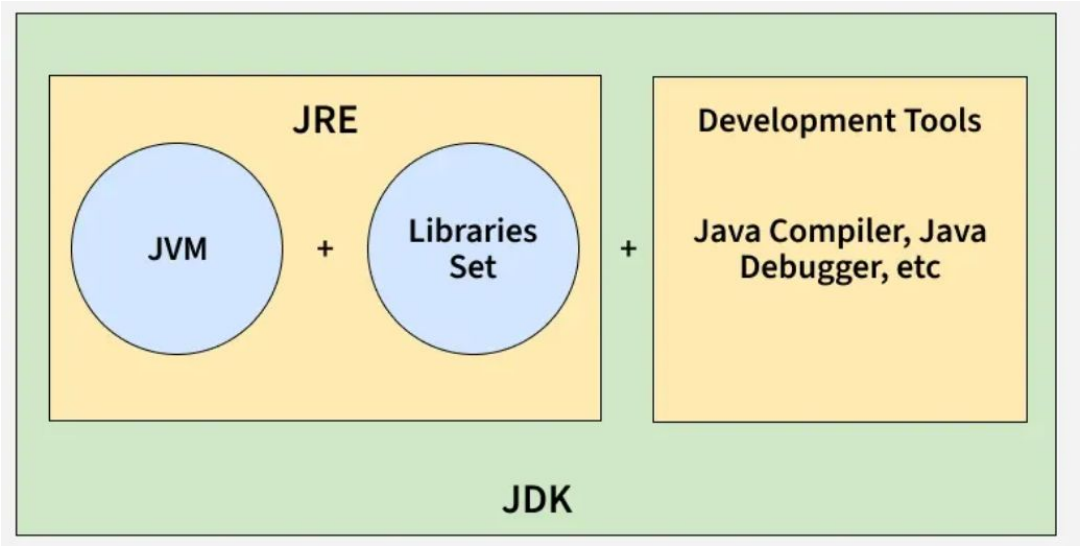


Java Development Kit (JDK)

- JDK (Java Development Kit) is a **software package** that **provides all the tools required to develop, compile, debug, and run Java applications.**
- It is mainly used by Java programmers to create Java programs.
- It contains;
  - **Java Runtime Environment (JRE)**
  - **Java compiler (javac)**
  - **Debugger**
  - **Documentation tools**
  - **Other utilities required to develop and execute Java programs.**

Relationship between JDK, JRE, JVM, Java Libraries, and Development Tools:

- **JDK** – The complete software package required to develop, compile, debug, and run Java applications.
  - **JDK = JRE + Development Tools**
- **JRE** - provides everything needed to run Java programs. ie, **JRE = JVM + Java Libraries**



- The **JVM** is responsible for:
  - Loading Java bytecode
  - Verifying the bytecode
  - Converting bytecode into machine code
  - Executing the program
- The same Java program can run on Windows, Linux, or macOS without modification because of JVM.

- **Java Libraries -** pre-written classes and packages provided by Java that programmers can use directly.
  - java.lang
  - java.util
  - java.io
  - java.net

  These libraries save programming time by providing ready-made functionality.
- **Development Tools -**
  - These tools include:
    - javac - Compiles Java source code into bytecode
    - java - Runs Java programs
    - jdb - Debugs Java programs
    - javadoc - Generates documentation
    - jar - Creates Java Archive (.jar) files
  - These tools help programmers write, test, debug, and package Java applications.

JDK v/s JRE:

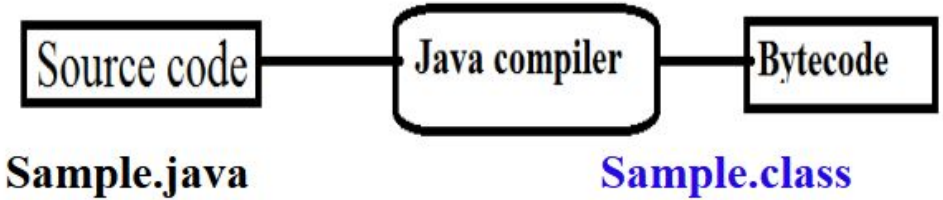
Feature	JDK	JRE
 Purpose	Used for developing Java programs	Used for running Java programs
 Compiler	Includes compiler (javac)	Does not include compiler
 Includes	Includes JRE	Does not include JDK
 Used By	Used by developers	Used by end users

JAVA COMPILER

- A Java compiler is a compiler for Java programming language.
- Java programs are compiled using javac command.
- Command for compilation

javac Program_name.java

- The output of compiling the java code is not executable code.
- It is called **bytecode** (Program_name.class)



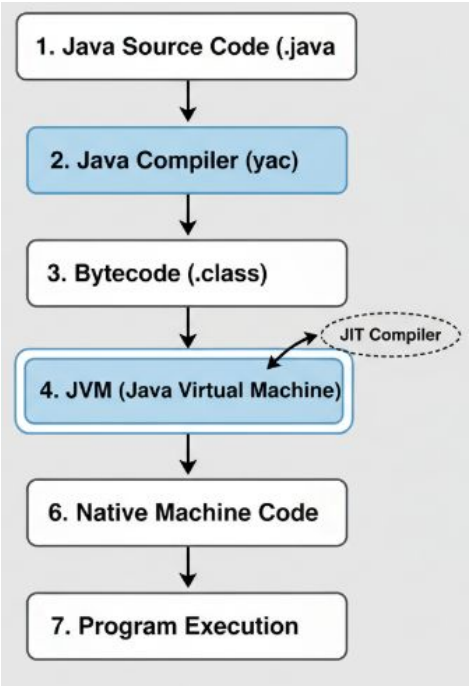
JAVA's MAGIC : THE BYTECODE

- Bytecode is a highly optimized **set of instructions designed to be executed by JVM,** which is a part of the JRE.
- **Translating a java program into bytecode makes it much easier to run a program in a variety of environments because only the JVM needs to be implemented for each platform.**
- Once a JRE exist for a given system, any Java program can run on it.
- Although the details of the JVM will differ from platform to platform, all understand the same Java bytecode.
- If a java program were compiled to native code, then different versions of the same program would have to exist for each type of CPU connected to the internet.

JIT Compiler:



- The JIT (Just-In-Time) Compiler in Java is used to improve the execution speed of Java programs.
- When a Java program is compiled, it is not converted directly into machine code.



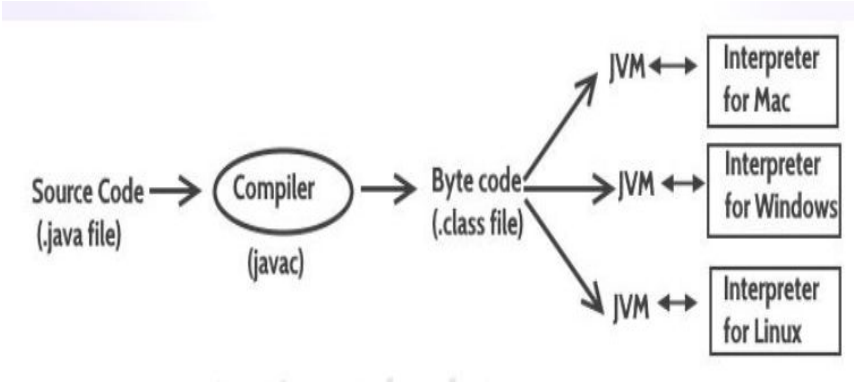
- The JVM can execute bytecode line by line using an interpreter, but interpreting every instruction repeatedly is slow. Java is a compiled interpreted language.
- The JIT Compiler solves this problem.

Working of JIT Compiler:

- JVM starts executing the bytecode using the interpreter.
- The JVM monitors which methods are executed frequently .
- **The JIT Compiler converts these frequently used bytecode methods into native machine code.**
- **The compiled machine code is stored in memory.**
- **The next time the same method is called, the JVM executes the machine code directly** instead of interpreting the bytecode again.

JAVA VIRTUAL MACHINE (JVM)

- JVM is an **abstract machine** that **converts Java bytecode into machine code** and **executes it on the host computer**.
- It **provides platform independence (WORA)** by allowing the same bytecode to run on any system that has a JVM.

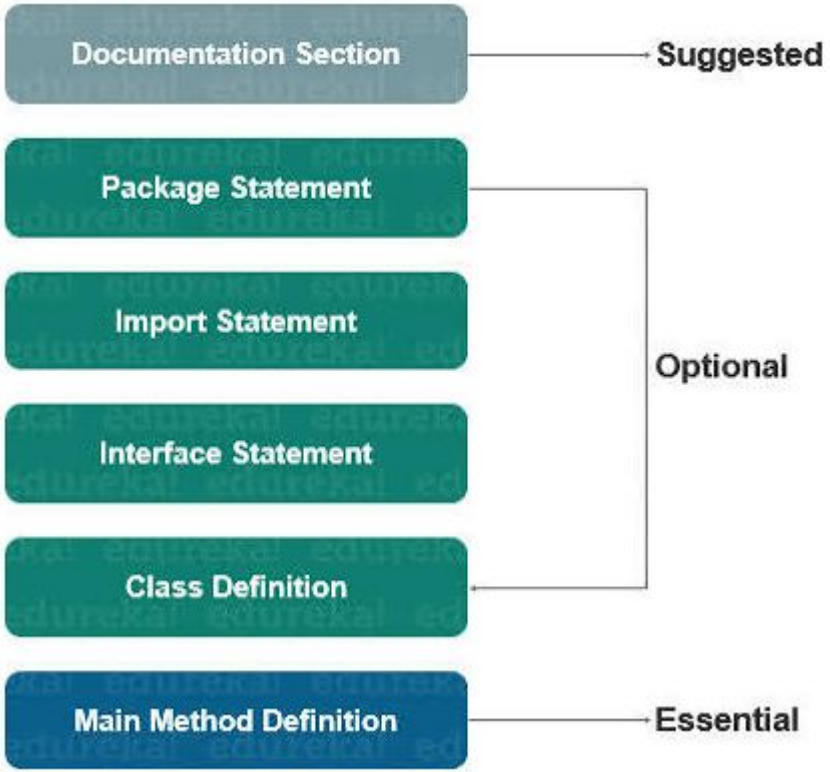


Note:

- *Abstract machine is a virtual (imaginary) computer that is defined by software rather than physical hardware. It does not physically exist.*

- When we compile a Java program, we get .class file(bytecode) which is not executable.
- JVM interprets the .class file into machine code depending on the operating system and hardware.
- JVM executes java programs like a machine.
- JVM is also responsible for garbage collection, array bound checking etc.
- JVM is platform independent.

STRUCTURE OF A JAVA PROGRAM



- A Java program is organized into packages, imports, classes, methods, statements, and comments.
- **Every Java application contains at least one class and a main() method,** which is the entry point of the program.

Documentation Section:

- We can write comments in this section for documentation.
- Comments are used to explain the program.
- They are ignored by the compiler.
- Java supports 3 types of comments;
  - **// Single-line comment**
  - **/* Multi-line**

    comment */

  - **/** Documentation comment */**

Package statement:

- A package is a **group of classes or interfaces.**
- A package is declared as;

  package package_name;

  - Eg: package pkg;
- We can create package with any name.
- If package statement is not there in the program, then program will be in default package.
- Package statement is optional.

Import statement:

- An import statement in Java is **used to make classes and interfaces from other packages available in your program.**
- Syntax:

  import packagename.*; *//means import all public classes and interfaces from the specified package.*

  import packagename.classname; *//This statement imports one specific class from a package*

- Eg:

  import java.util.Scanner; *//This statement imports the Scanner class from the java.util package.*

  import java.io.*; *//imports all public classes and interfaces from the java.io package.*

  - import → Keyword used to import a class or package.
  - java → Top-level package.
  - util → Utility package containing useful classes.
  - Scanner → Class used to read input from the keyboard, files, or other input sources.

Interface statement:

- Used to specify interface in Java.
- **An interface in Java is a blueprint of a class.**
- **It contains a group of method declarations (without implementation) and constants.**
- It is an optional section.

Class Definition:

- Classes are the **main and essential elements of any Java program.**
- It is a common convention to **start every word of class name with capital letter.**
- A **class** in Java is a **blueprint or template from which objects are created.**
- An **object** is **an instance of a class.**
- Class **defines the data (variables) and behavior (methods)** that objects of the class will have.

Syntax:

```
class class_name
{
 // Data members
// Constructors
// Methods
}
```

- A class can contain:
  - Fields (instance variables)
  - Constructors
  - Methods
  - Nested classes (optional)

Example:

```
class Student
{
int rollNo = 101;
String name = "Anu";

void display()
      {
      System.out.println("Roll Number: " + rollNo);
      System.out.println("Name: " + name);
      }
}
```

Main method Definition:

- The main() method is the **entry point of a Java application.**
- The Java Virtual Machine **(JVM) starts program execution by invoking the main() method.**
- The main() method does not return a value (void).
- Syntax:

```
public static void main(String[] args)

{

    // Statements

}
```

Keyword	Meaning
public	The method is **accessible from anywhere** so that the JVM can call it.
static	Allows the **method to be called without creating an object** of the class.
void	Indicates that the **method does not return any value.**
main	The predefined method name recognized by the JVM as the **program's entry point.**
String args[ ]	An **array of String objects** used to receive command-line arguments.

PYQ's:

1. Why is the 'main' method in java qualified as public, static and void?
2. Outline the reasons of why Java programs known as "Write once and run anywhere" programs?
3. What is Just-in-time compiler?
4. What is the role of JVM?
5. A java program is compiled on a windows machine and .class file is transferred to a Linux machine. The Linux machine does not have the JDK installed, only the JRE. Can the program be executed on the Linux machine? Justify your answer.
6. What is the role of the JIT compiler in JVM? Explain how it improves the performance of Java programs during execution.
7. Explain bytecode and JDK.

Simple Java program to print "Hello, World!"

```
public class HelloWorld
{
    public static void main(String[] args)
        {
        System.out.println("Hello, World!");
        }
}
```

Commands to compile & run:

javac HelloWorld.java

java HelloWorld

Output:

Hello, World!

Java program to add two numbers:

```
public class AddTwoNumbers
{
    public static void main(String[] args)
        {
       int a = 10;
       int b = 20;
       int sum;
       sum = a + b;
       System.out.println("First Number = " + a);
       System.out.println("Second Number = " + b);
       System.out.println("Sum = " + sum);
        }
}
```

Commands to compile & run:

javac AddTwoNumbers.java

java AddTwoNumbers

Output:

First Number = 10

Second Number = 20

Sum = 30

Program to demonstrate + operator in Java:

```
public class Sum{
   public static void main(String[] args){
      int num1 = 10;
      int num2 = 20;
      int sum = num1+num2;
      System.out.println(sum);
      System.out.println(num1+num2);
      System.out.println(10+20);
      System.out.println("Sum is:"+10+20);
      System.out.println("Sum is:"+num1+num2);
      System.out.println("Sum is:"+sum);
      System.out.println("Sum is:"+10+20);
      System.out.println("Sum is:"+10+20+"30");
      System.out.println("Sum is:"+(10+20));
         }}
```

OUTPUT:

```
30
30
30
Sum is:1020
Sum is:1020
Sum is:30
Sum is:1020
Sum is:102030
Sum is:30
```

Why is Java a Strongly Typed Language?

- In Java, **every variable has a type, every expression has a type, and every type is strictly defined.**
- **All assignments**, whether explicit or via parameter passing in method calls, **are checked for type compatibility.**
- **No automatic conversions of conflicting types** are possible.
- The **Java compiler checks all expressions and parameters to ensure that the types are compatible.**
    - ie, type checking is performed at compile time.
- Any type mismatches are errors that must be corrected before the compiler will finish compiling the class.

DATA TYPES IN JAVA

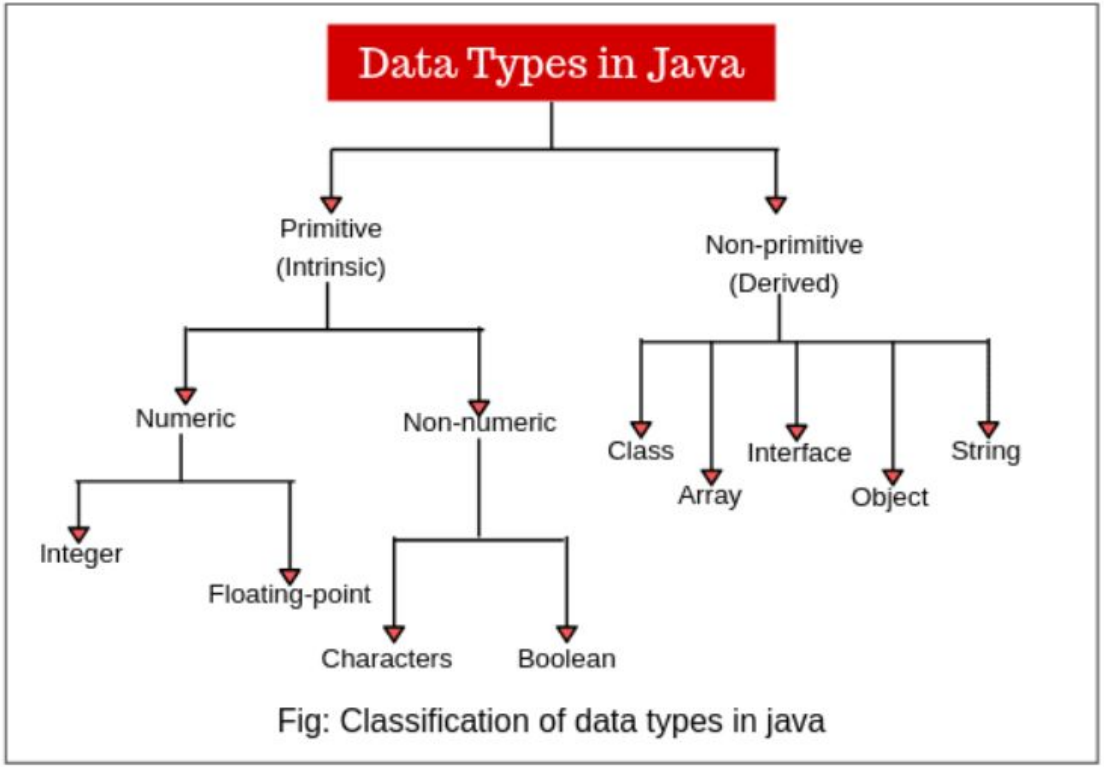
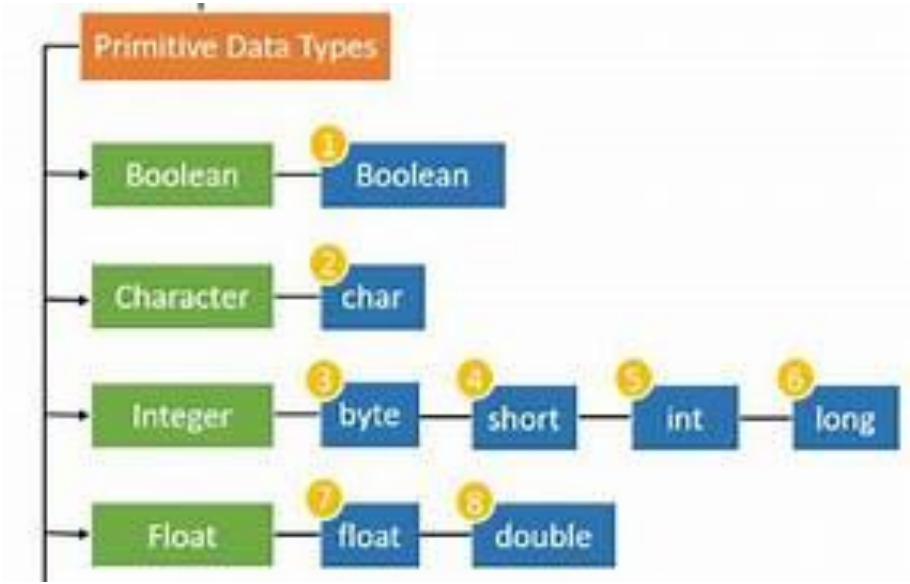


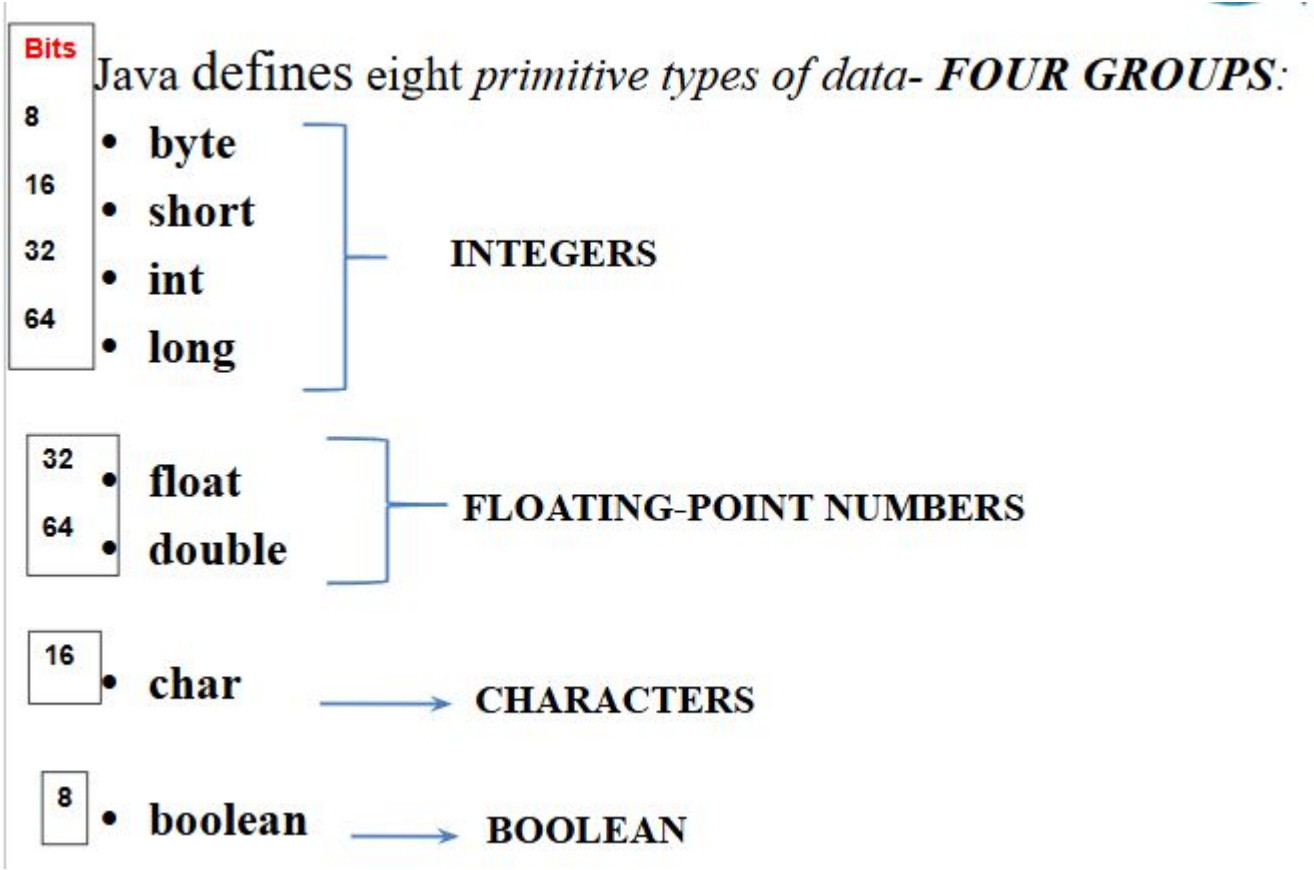
Fig: Classification of data types in java

PRIMITIVE DATA TYPES IN JAVA:

- Primitive data types are the **built-in data types provided by Java** to **store simple values** such as numbers, characters, and logical values.



- Java has **8 primitive data types**.
- Primitive data types store actual values, not objects.
- They are fast and memory-efficient.
- **int** is the default choice for integers.
- **double** is the default choice for decimal numbers.
- **char** stores a single Unicode character.
- **boolean** stores either true or false.


Bits
Java defines eight *primitive types of data*- ***FOUR GROUPS***:
8
16
32
64
• byte
• short
• int
• long
INTEGERS
32
64
• float
• double
FLOATING-POINT NUMBERS
16
• char
CHARACTERS
8
• boolean
BOOLEAN

Integers:

- This group includes;
  - **byte (8 bits)**
  - **short (16 bits)**
  - **int (32 bits)**
  - **long (64 bits)**

  For whole-valued signed numbers
- Can be **signed, positive or negative values.**
- Java does not support unsigned, positive-only integers.
- The width of an integer type is the behavior it defines for variables and expressions of that type.

byte:

- The **smallest integer type** is byte.
- This is a signed **8-bit** type
- It has a range from –128 to 127.
- Useful when working with a stream of data from a network or file.
- Eg:

  byte b, c;

  - The above statement declares two byte variables called b and c.

short:

- short is a signed **16-bit type.**
- It has a range from –32,768 to 32,767.
- It is the **least-used** Java type.
- Eg:

  short s;

  short t;

int:

- **32-bit** type.
- Variables of type int are **commonly employed;**
  - to control loops
  - to index arrays
- When byte and short values are used in an expression they are promoted to int when the expression is evaluated.
- int is often the **best choice** when an integer is needed.

long:

- long is a signed **64-bit type.**
- Used in occasions where an int type is not large enough to hold the desired value.
- The range of a long is quite large.

Floating-point types:

- Also known as **real numbers.**
- They are used when evaluating expressions that require **fractional precision.**
  - Eg: Calculations such a square root, sine, cosine etc.,
- There are two types of floating point types;
  - **float** => 32 bits
  - **double** =>64 bits

float:

- Single-precision value that uses **32 bits** of storage.
- Faster on some processors and takes half as much space as double precision, but will become imprecise when the values are either very large or very small.
- float variables are useful when we **need a fractional component, but don't require a large degree of precision.**
- Eg: float hightemp,lowtemp;

double:

- Uses **64-bits** to store a value.
- Double precision is actually faster than single precision on some modern processors.
- All transcendental math functions, such as sin( ), cos( ), and sqrt( ), return double values.
- When we need to maintain accuracy over many iterative calculations, or are manipulating large valued numbers, double is the best choice.

Character:

- In Java, the data type used to store characters is **char.**
- char in Java is not the same as char in C or C++.
  - In C/C++, char is 8 bits wide.
- Java uses **Unicode** to represent characters.
  - Unicode defines a fully international character set that can represent all of the characters found in all human languages.
- So it requires **16 bits.**
  - The range of a char is 0 to 65,536.
- There are **no negative chars.**
- Eg:

```
char ch = ‘a’;
```

```
// Demonstrate char data type.
class CharDemo
{
    public static void main(String args[])
    {
    char ch1, ch2;
    ch1 = 88;        // code for X
    ch2 = 'Y';
    System.out.print("ch1 and ch2: ");
    System.out.println(ch1 + " " + ch2);
    }
}
```

- *Note:*
  - *ch1 is assigned the value 88, which is the ASCII (and Unicode) value that corresponds to letter X.*

```
OUTPUT
ch1 and ch2:X Y
```

Booleans:

- Java has a primitive type, called boolean, **for logical values.**
- It can have only one of two possible values, **true or false.**
- This is the **type returned by all relational operators.**
- boolean is also the **type required by the conditional expressions that govern the control statements** such as if and for.

```
// Demonstrate boolean values.
class BoolTest
{
    public static void main(String args[]) {
    boolean b;
    b = false;
    System.out.println("b is " + b);
    b = true;
    System.out.println("b is " + b);
    if(b)
       System.out.println("This is executed.");
    b = false;
    if(b)
       System.out.println("This is not executed.");
    System.out.println("10 > 9 is " + (10 > 9));
    } }
```

OUTPUT

b is false
b is true
This is executed.
10 > 9 is true

WRAPPER TYPES IN JAVA

- Java uses primitive types for simple values.
  - Eg: 12 represents int.
    2.5 represents float.
- In some cases, we have to **represent values as objects.**
  - With primitive types, we can't represent values as objects.
  - Primitive types cannot be passed as reference to methods.
  - Many data structures operate on objects.
    - Java provides type wrappers for this.
- Wrapper classes in Java are **classes that wrap (encapsulate) primitive data types into objects.**

Why Do We Need Wrapper Classes?

- Primitive data types are not objects, so they cannot be used where Java expects objects.
- Wrapper classes are useful because they:
  - Allow primitive values to be treated as objects.
  - Are required for Collections like ArrayList, HashMap, etc.
  - Provide useful methods for conversion and comparison.
  - Support autoboxing and unboxing.
    - **Autoboxing** : The automatic conversion of a **primitive value into its corresponding wrapper object.**
    - **Unboxing** : The automatic conversion of a **wrapper object into its corresponding primitive value.**

- Java provides one wrapper class for each primitive data type.

Primitive Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Character wrapper

- Character is a wrapper around a char.
- The constructor for Character is;

```
Character(char ch)
```

- To obtain the char value contained in a Character object, call charValue( ):

  char charValue( )

  - It returns the encapsulated character.

Boolean wrapper

- Boolean is a wrapper around boolean values.
- It defines these constructors:

  Boolean(boolean boolValue)

  Boolean(String boolString)

- To obtain a boolean value from a Boolean object, use booleanValue( ), :

  boolean booleanValue( )

  - It returns the boolean equivalent of the invoking object.

The Numeric Type Wrappers:

- Byte, Short, Integer, Long, Float, and Double => Numeric type wrappers
- It defines these constructors:

  Integer(int num)

  Integer(String str)

- All of the numeric type wrappers inherit the abstract class Number.
- All of the numeric type wrappers **define constructors that allow an object to be constructed from a given value, or a string representation of that value.**

- Functions are:

```
byte byteValue( )          // returns the value of an object as a byte
double doubleValue( )      // returns the value of an object as a double
float floatValue( )
int intValue( )
long longValue( )
short shortValue( )
```

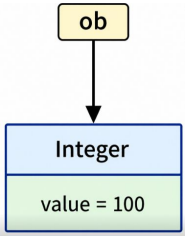
Example Program : Integer Wrapper

```
class Wrap
 {
    public static void main(String args[])
      {
      Integer ob = new Integer(100);
      int i = ob.intValue(); //Unboxing
      System.out.println(i + " " + ob);
      }
}
```

OUTPUT:

100 100

- Integer is the wrapper class for the primitive type int.
- new Integer(100) creates an Integer object containing the value 100.
- The reference variable *ob* points to this object.



- intValue() is a method of the Integer class.
    - It extracts the primitive int value stored inside the wrapper object.
- i => 100 (primitive int)
- ob => 100 (Integer object displayed as a string)

Primitive vs Wrapper Class

Primitive Data Type	Wrapper Class
Stores simple values	Stores values as objects
Faster	Slightly slower (object overhead)
Cannot be used in Collections	Can be used in Collections
No methods	Provides many useful methods
Memory efficient	Uses more memory

CASTING & AUTOBOXING IN JAVA:

Casting:

- **Casting is the process of converting one data type into another.**
- Types of Casting:
  1. **Implicit Casting (Widening Casting)**
     - Converting a smaller data type into a larger data type automatically.
     - Eg:

```
int num = 100;

double d = num;   // int → double

System.out.println(d);
```

OUTPUT: 100.0

2. Explicit Casting (Narrowing Casting)

- Converting a larger data type into a smaller data type manually.
- Eg:

```
double d = 99.99;

int num = (int)d;   // double → int

System.out.println(num);
```

OUTPUT: 99

- The fractional part is lost during narrowing.

Autoboxing:

- Autoboxing is the **automatic conversion of a primitive data type into its corresponding wrapper object.**
- Eg:

```
int x = 50;

Integer obj = x;   // Autoboxing

System.out.println(obj);
```

OUTPUT: 50

Primitive Value int x = 50 —-----> Autoboxing —------> Integer obj = 50

Casting vs Autoboxing

Casting	Autoboxing
Converts one data type to another	Converts primitive to wrapper object
Used between primitive types	Used between primitive and wrapper classes
May be implicit or explicit	Always automatic
Example: int → double	Example: int → Integer

OPERATORS IN JAVA

- Java provides a rich operator environment.
- **Operators are special symbols used to perform operations on variables and values.**
- They are the **building blocks of expressions** in Java.
- Java groups operators into categories such as;
  - **Arithmetic operators**
  - **Bitwise operators**
  - **Relational operators**
  - **Logical operators**
  - **Assignment operator**
  - **The ? operator (Conditional Operator)**

Refer lecture notes for detailed overview of operators.

CONTROL STATEMENTS IN JAVA

- Control statements are used to **control the flow of execution of a Java program.**
- By default, Java executes statements one after another (sequentially).
- Without control statements, a program would execute every statement in order, regardless of the situation.
- Control statements make programs dynamic, flexible, and intelligent by allowing them to respond to different inputs and conditions.
- Java's program control statements can be put into the following categories:
  - **Selection statements** - allow the program to choose different paths of execution based on the outcome of an expression or the state of a variable
  - **Iteration statements** - enable program execution to repeat one or more statements.
  - **Jump statements** - allow the program to execute in a nonlinear fashion.

Selection statements:

- Java supports two selection statements:
  - **if**
  - **switch**
- These statements allow to control the flow of your program's execution based upon conditions known only during run time.

if :

- The if statement is Java's conditional branch statement.
- It can be used to route program execution through two different paths.

General form:

```
if (condition) statement1;
else statement2;
```

- Here, each statement may be a single statement or a compound statement enclosed in curly braces.
- The condition is any expression that returns a boolean value.
- The else clause is optional.

Working:

- If the condition is true, then statement1 is executed.
- Otherwise, statement2 (if it exists) is executed.
- In no case will both statements be executed.

Example:

```
class IfExample
{
   public static void main(String args[])
   {
      int num = 10;
      if (num > 0)
       {
         System.out.println("The number is positive.");
       }
      System.out.println("Program completed.");
   }
}
```

Working:

- If the condition is true, then statement1 is executed.
- Otherwise, statement2 (if it exists) is executed.
- In no case will both statements be executed.

Nested ifs

- A nested if is an if statement that is the target of another if or else.
- Nested ifs are very common in programming.
- In nested ifs, an else statement always refers to the nearest if statement that is within the same block as the else and that is not already associated with an else.

Example:

```
if(i == 10){

if(j < 20) a = b;

if(k > 100) c = d; // this if is

else a = c; // associated with this else

}

else a = d; // this else refers to if(i == 10)
```

- *As the comments indicate, the final else is not associated with if(j<20) because it is not in the same block (even though it is the nearest if without an else).*
- *Rather, the final else is associated with if(i==10).*
- *The inner else refers to if(k>100) because it is the closest if within the same block.*

The if-else-if Ladder:

- A common programming construct that is based upon a sequence of nested ifs is the if-else-if ladder.
- It looks like this:

```
if(condition)
  statement;
else if(condition)
  statement;
else if(condition)
  statement;
.
.
.
else
  statement;
```

- The if statements are executed from the top down.
- As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed.
- If none of the conditions is true, then the final else statement will be executed.
- The final else acts as a default condition; that is, if all other conditional tests fail, then the last else statement is performed.
- If there is no final else and all other conditions are false, then no action will take place.

```
// Demonstrate if-else-if statements.
class IfElse {
  public static void main(String args[]) {
    int month = 4; // April
    String season;

    if(month == 12 || month == 1 || month == 2)
      season = "Winter";
    else if(month == 3 || month == 4 || month == 5)
      season = "Spring";
    else if(month == 6 || month == 7 || month == 8)
      season = "Summer";
    else if(month == 9 || month == 10 || month == 11)
      season = "Autumn";
    else
      season = "Bogus Month";

    System.out.println("April is in the " + season + ".");
  }
}
```

Output:

```
// Demonstrate if-else-if statements.
class IfElse {
  public static void main(String args[]) {
    int month = 4; // April
    String season;

    if(month == 12 || month == 1 || month == 2)
      season = "Winter";
    else if(month == 3 || month == 4 || month == 5)
      season = "Spring";
    else if(month == 6 || month == 7 || month == 8)
      season = "Summer";
    else if(month == 9 || month == 10 || month == 11)
      season = "Autumn";
    else
      season = "Bogus Month";

    System.out.println("April is in the " + season + ".");
  }
}
```

Output:

April is in the Spring.

switch:

- The switch statement is Java's multiway branch statement.
- It provides an easy way to dispatch execution to different parts of the code based on the value of an expression.
- switch often provides a better alternative than a large series of if-else-if statements.

General form of a switch statement:

```
switch (expression) {
  case value1:
    // statement sequence
    break;
  case value2:
    // statement sequence
    break;
.
.
.
  case valueN:
    // statement sequence
    break;
  default:
    // default statement sequence
}
```

- *The expression must be of type byte, short, int, or char.*
- *Each of the values specified in the case statements must be of a type compatible with the expression.*
- *Each case value must be a unique literal (that is, it must be a constant, not a variable).*
- *Duplicate case values are not allowed.*

Working:

- **The value of the expression is compared with each of the literal values in the case statements.**
  - **If a match is found, the code sequence following that case statement is executed.**
  - **If none of the constants matches the value of the expression, then the default statement is executed.**
- The default statement is optional.
- If no case matches and no default is present, then no further action is taken.
- The break statement is used inside the switch to terminate a statement sequence.
  - When a break statement is encountered, execution branches to the first line of code that follows the entire switch statement.
  - This has the effect of “jumping out” of the switch.

Example: Switch statement:

```
// A simple example of the switch.
class SampleSwitch {
  public static void main(String args[]) {
    for(int i=0; i<6; i++)
      switch(i) {
        case 0:
          System.out.println("i is zero.");
          break;
        case 1:
          System.out.println("i is one.");
          break;
        case 2:
          System.out.println("i is two.");
          break;
        case 3:
          System.out.println("i is three.");
          break;
        default:
          System.out.println("i is greater than 3.");
      }
  }
}
```

OUTPUT:

Example: Switch statement:

```
// A simple example of the switch.
class SampleSwitch {
  public static void main(String args[]) {
    for(int i=0; i<6; i++)
      switch(i) {
        case 0:
          System.out.println("i is zero.");
          break;
        case 1:
          System.out.println("i is one.");
          break;
        case 2:
          System.out.println("i is two.");
          break;
        case 3:
          System.out.println("i is three.");
          break;
        default:
          System.out.println("i is greater than 3.");
      }
  }
}
```

OUTPUT:

```
i is zero.
i is one.
i is two.
i is three.
i is greater than 3.
i is greater than 3.
```

Iteration statements:

- Java's iteration statements are;
  - **for**
  - **while**
  - **do-while**
- These statements create what we commonly call **loops.**
  - **A loop repeatedly executes the same set of instructions until a termination condition is met.**

while:

- The while loop is Java's most fundamental loop statement.
- **It repeats a statement or block while its controlling expression is true.**

General form:

```
while(condition) {
   // body of loop
}
```

- The condition can be any Boolean expression.
- The body of the loop will be executed as long as the conditional expression is true.
- When condition becomes false, control passes to the next line of code immediately following the loop.
- The curly braces are unnecessary if only a single statement is being repeated.

Example:

```
// Demonstrate the while loop.
class While {
  public static void main(String args[]) {
    int n = 10;

    while(n > 0) {
      System.out.println("tick " + n);
      n--;
    }
  }
}
```

OUTPUT:

Example:

```
// Demonstrate the while loop.
class While {
  public static void main(String args[]) {
    int n = 10;

    while(n > 0) {
      System.out.println("tick " + n);
      n--;
    }
  }
}
```

OUTPUT:

```
tick 10
tick 9
tick 8
tick 7
tick 6
tick 5
tick 4
tick 3
tick 2
tick 1
```

do-while:

- In while loop, if the conditional expression is initially false => then the body of the loop will not be executed at all.
- At times, it is desirable to execute the body of a loop at least once, even if the conditional expression is false to begin with.
  - ie, there are times when you would like to test the termination expression at the end of the loop rather than at the beginning.
  - Fortunately, Java supplies such a loop - the do-while.
  - **The do-while loop always executes its body at least once, because its conditional expression is at the bottom of the loop.**

General form:

```
do {
  // body of loop
} while (condition);
```

- Each iteration of the do-while loop first executes the body of the loop and then evaluates the conditional expression.
- If this expression is true, the loop will repeat.
- Otherwise, the loop terminates.

Example:

```
// Demonstrate the do-while loop.
class DoWhile {
  public static void main(String args[]) {
    int n = 10;

    do {
      System.out.println("tick " + n);
      n--;
    } while(n > 0);
  }
}
```

OUTPUT:

```
tick 10
tick 9
tick 8
tick 7
tick 6
tick 5
tick 4
tick 3
tick 2
tick 1
```

for

General form:

```
for(initialization; condition; iteration) {
  // body
}
```

Working:

- When the loop first starts, the **initialization portion** of the loop is executed.
  - Generally, this is an expression that sets the value of the loop control variable, which acts as a counter that controls the loop.
- Next, **condition** is evaluated.
  - This must be a Boolean expression.
  - It usually tests the loop control variable against a target value.
  - If this expression is true, then the body of the loop is executed.
  - If it is false, the loop terminates.
- Next, the **iteration** portion of the loop is executed.
  - This is usually an expression that increments or decrements the loop control variable.
  - The loop then iterates, first evaluating the conditional expression, then executing the body of the loop, and then executing the iteration expression with each pass.
  - This process repeats until the controlling expression is false.

Example:

```
// Demonstrate the for loop.
class ForTick {
  public static void main(String args[]) {
    int n;

    for(n=10; n>0; n--)
      System.out.println("tick " + n);
  }
}
```

OUTPUT:

```
tick 10
tick 9
tick 8
tick 7
tick 6
tick 5
tick 4
tick 3
tick 2
tick 1
```

Note:

<u>Using the Comma</u>

- *There will be times when we want* ***to include more than one statement in the initialization and iteration portions of the for loop.***
- *To allow two or more variables to control a for loop,* ***Java permits you to include multiple statements in both the initialization and iteration portions*** *of the for.*
- *Each statement is separated from the next by a* ***comma.***

```
for(a=1, b=4; a<b; a++, b--) {
  System.out.println("a = " + a);
  System.out.println("b = " + b);
}
```

THIS IS VALID IN JAVA

Jump statements:

- Java supports three jump statements:
  - **break**
  - **continue**
  - **return**
- These statements **transfer control to another part of your program.**

break:

- In Java, the break statement has three uses.
  - First, **it terminates a statement sequence in a switch statement.**
  - Second, **it can be used to exit a loop.**
  - Third, it can be **used as a “civilized” form of goto.**

Using break to Exit a Loop:

- By using break, we can **force immediate termination of a loop,** bypassing the conditional expression and any remaining code in the body of the loop.
- When a break statement is encountered inside a loop, **the loop is terminated and program control resumes at the next statement following the loop**.

Example:

```
// Using break to exit a loop.
class BreakLoop {
  public static void main(String args[]) {
    for(int i=0; i<100; i++) {
      if(i == 10) break; // terminate loop if i is 10
      System.out.println("i: " + i);
    }
    System.out.println("Loop complete.");
  }
}
```

OUTPUT:

```
i: 0
i: 1
i: 2
i: 3
i: 4
i: 5
i: 6
i: 7
i: 8
i: 9
Loop complete.
```

Using break as a Form of Goto:

- break statement can also be employed by itself to provide a “civilized” form of the goto statement.
- **Java does not have a goto statement** because it provides a way to branch in an arbitrary and unstructured manner.
- Instead, **java defines an expanded form of the break statement.**
  - By using this form of break, we can, for example, break out of one or more blocks of code
  - These blocks need not be part of a loop or a switch.
  - They can be any block.
  - Further, we can specify precisely where execution will resume, because **this form of break works with a label.**

General form:

break label;

- label is the name of a label that identifies a block of code.
- This can be a stand-alone block of code but it can also be a block that is the target of another statement.
- When this form of break executes, control is transferred out of the named block.
- To name a block, put a label at the start of it.
- A label is any valid Java identifier followed by a colon.
- Once you have labeled a block, you can then use this label as the target of a break statement.
- Doing so causes execution to resume at the end of the labeled block.

Using continue:

- The continue statement is **used inside loops to skip the current iteration and move directly to the next iteration of the loop.**
- A continue statement is most useful when we want to skip certain iterations of a loop without stopping the entire loop.
- In while and do-while loops, a continue statement causes control to be transferred directly to the conditional expression that controls the loop.
- In a for loop, control goes first to the iteration portion of the for statement and then to the conditional expression.
- For all three loops, any intermediate code is bypassed.

Example:

```
// Demonstrate continue.
class Continue {
  public static void main(String args[]) {
    for(int i=0; i<10; i++) {
      System.out.print(i + " ");
      if (i%2 == 0) continue;
      System.out.println("");
    }
  }
}
```

OUTPUT:

```
0 1
2 3
4 5
6 7
8 9
```

return:

- The return statement is **used to explicitly return from a method**.
- At any time in a method, **the return statement can be used to cause execution to branch back to the caller of the method**.
- Thus, the return statement immediately terminates the method in which it is executed.

Example:

```
// Demonstrate return.
class Return {
  public static void main(String args[]) {
    boolean t = true;

    System.out.println("Before the return.");

    if(t) return; // return to caller

    System.out.println("This won't execute.");
  }
}
```

OUTPUT:

Before the return.

- The final println( ) statement is not executed.
- As soon as return is executed, control passes back to the caller

CLASSES

- A class is a **blueprint or template used to create objects.**
- It defines a new data type.
  - Once defined, this new type can be used to create objects of that type.
  - Thus, a **class is a template for an object**, and an **object is an instance of a class.**
- Class **defines the properties (data) and behaviors (methods) that objects of that class will have.**

The General Form of a Class:

- A class is declared by use of the **class** keyword.
- The data, or variables, defined within a class are called **instance variables.**
- The code is contained within **methods.**
- Collectively, the methods and variables defined within a class are called members of the class.
- In most classes, the instance variables are acted upon and accessed by the methods defined for that class.

The general form of a class definition:

```
class classname {
    type instance-variable1;
    type instance-variable2;
    // ...
    type instance-variableN;

    type methodname1(parameter-list) {
      // body of method
    }
    type methodname2(parameter-list) {
      // body of method
    }
    // ...
    type methodnameN(parameter-list) {
      // body of method
    }
}
```

Example:

```
class Student
{
    int rollNo;
    String name;
    void display()
        {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
         }
}
```

OBJECTS:

- An object is an **instance of a class**.
- It represents a real-world entity and **occupies memory.**
- Objects are created using the **new keyword.**

Declaring Objects:

- When we create a class, we are creating a new data type.
- We can use this type to declare objects of that type.
- The **new** operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it.
- In Java, all class objects must be dynamically allocated.

Syntax:

```
ClassName objectName = new ClassName();
```

A Simple Class

```
/* A program that uses the Box class.

   Call this file BoxDemo.java
*/
class Box {
  double width;
  double height;
  double depth;
}

// This class declares an object of type Box.
class BoxDemo {
  public static void main(String args[]) {
    Box mybox = new Box();
    double vol;

    // assign values to mybox's instance variables
    mybox.width = 10;
    mybox.height = 20;
    mybox.depth = 15;

    // compute volume of box
    vol = mybox.width * mybox.height * mybox.depth;

    System.out.println("Volume is " + vol);
  }
}
```

- To run this program, we must execute BoxDemo.class. Because the main( ) method
- is in the class called BoxDemo
- Output:

```
Volume is 3000.0
```

- Here, Box is a class that defines three instance variables: *width, height*, and *depth*.
- A class defines a new type of data.
- In this case, the new data type is called Box.
- Box mybox = new Box(); // create a Box object called mybox
- After this statement executes, mybox will be an instance of Box.
- Thus, it will have “physical” reality.
- mybox.width = 100; // assign the width variable of mybox the value 100.

The dot (.) operator:

- **The dot operator (.) in Java is used to access members (variables and methods) of a class or object.**
- **It tells Java "go inside this object or class and access the specified member."**
- **Each time we create an instance of a class, we are creating an object that contains its own copy of each instance variable defined by the class.**
- Thus, every Box object will contain its own copies of the instance variables width, height, and depth.
- To access these variables, we can use the dot (.) operator.
- The dot operator links the name of the object with the name of an instance variable.

```
// This program declares two Box objects.

class Box {
  double width;
  double height;
  double depth;
}

class BoxDemo2 {
  public static void main(String args[]) {
    Box mybox1 = new Box();
    Box mybox2 = new Box();
    double vol;

    // assign values to mybox1's instance variables
    mybox1.width = 10;
    mybox1.height = 20;
    mybox1.depth = 15;

    /* assign different values to mybox2's
       instance variables */
    mybox2.width = 3;
    mybox2.height = 6;
    mybox2.depth = 9;

    // compute volume of first box
    vol = mybox1.width * mybox1.height * mybox1.depth;
    System.out.println("Volume is " + vol);

    // compute volume of second box
    vol = mybox2.width * mybox2.height * mybox2.depth;
    System.out.println("Volume is " + vol);
  }
}
```

- This program declares two Box objects - **mybox1** and **mybox2**.
- As stated earlier, each object has its own copies of the instance variables.
- ie, if you have two Box objects, each has its own copy of depth, width, and height.
- It is important to understand that changes to the instance variables of one object have no effect on the instance variables of another.

OUTPUT:

```
Volume is 3000.0
Volume is 162.0
```

ADDING A METHOD TO THE BOX CLASS

```
// This program includes a method inside the box class.

class Box {
  double width;
  double height;
  double depth;

  // display volume of a box
  void volume() {
    System.out.print("Volume is ");
    System.out.println(width * height * depth);
  }

class BoxDemo3 {
  public static void main(String args[]) {
    Box mybox1 = new Box();
    Box mybox2 = new Box();

    // assign values to mybox1's instance variables
    mybox1.width = 10;
    mybox1.height = 20;
    mybox1.depth = 15;

    /* assign different values to mybox2's
       instance variables */
    mybox2.width = 3;
    mybox2.height = 6;
    mybox2.depth = 9;

    // display volume of first box
    mybox1.volume();

    // display volume of second box
    mybox2.volume();
  }
}
```

OUTPUT:

```
Volume is 3000.0
Volume is 162.0
```

RETURNING A VALUE:

```
class Box {
  double width;
  double height;
  double depth;

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}

class BoxDemo4 {
  public static void main(String args[]) {
    Box mybox1 = new Box();
    Box mybox2 = new Box();
    double vol;

    // assign values to mybox1's instance variables
    mybox1.width = 10;
    mybox1.height = 20;
    mybox1.depth = 15;

    /* assign different values to mybox2's
       instance variables */
    mybox2.width = 3;
    mybox2.height = 6;
    mybox2.depth = 9;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);
    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

OUTPUT:

```
Volume is 3000.0
Volume is 162.0
```

ADDING A METHOD THAT TAKES PARAMETERS:

```
// This program uses a parameterized method.

class Box {
  double width;
  double height;
  double depth;

  // compute and return volume
  double volume() {
    return width * height * depth;
  }

  // sets dimensions of box
  void setDim(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }
}

class BoxDemo5 {
  public static void main(String args[]) {
    Box mybox1 = new Box();
    Box mybox2 = new Box();
    double vol;

    // initialize each box
    mybox1.setDim(10, 20, 15);
    mybox2.setDim(3, 6, 9);

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

- the setDim( ) method is used to set the dimensions of each box.
- Eg:

  mybox1.setDim(10, 20, 15);

  - 10 is copied into parameter w, 20 is copied into h, and 15 is copied into d.
  - Inside setDim( ) the values of w, h, and d are then assigned to width, height, and depth, respectively.

OUTPUT:

```
Volume is 3000.0
Volume is 162.0
```

HOMEWORK:

1. Define "class" and "object". Write a Java class Employee with the fields empId, name, and salary. Include: A parameterized constructor. A method displayEmployee() to display employee details. A method calculateAnnualSalary() to return the annual salary. In the main() method: Create an Employee object. Display the employee details and annual salary.

2. Define "class" and "object". Write a Java class Car with the fields make, model, and year. Include: A parameterized constructor. A method displayCarDetails() to display the car details. A method calculateCarAge(int currentYear) to return the age of the car. In the main() method: Create a Car object.Display the car details. Display the age of the car by calling the appropriate method.

CONSTRUCTORS:

- A constructor **initializes an object immediately upon creation.**
- It has the **same name as the class** in which it resides and is **syntactically similar to a method.**
- **Once defined, the constructor is automatically called immediately after the object is created, before the new operator completes.**
- Constructors have **no return type.**
    - This is because the implicit return type of a class' constructor is the class type itself.
- It is the constructor's job to initialize the internal state of an object so that the code creating an instance will have a fully initialized, usable object immediately.

- We can rework the *Box example of page 106* so that the dimensions of a box are automatically initialized when an object is constructed.
- To do so, we can replace setDim( ) with a constructor.
- For that, we can define a simple constructor that simply sets the dimensions of each box to the same values. (As below)

```
/* Here, Box uses a constructor to initialize the
   dimensions of a box.
*/
class Box {
  double width;
  double height;
  double depth;

  // This is the constructor for Box.
  Box() {
    System.out.println("Constructing Box");
    width = 10;
    height = 10;
    depth = 10;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo6 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box mybox1 = new Box();
    Box mybox2 = new Box();

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

OUTPUT:

```
Constructing Box
Constructing Box
Volume is 1000.0
Volume is 1000.0
```

- Here , both mybox1 and mybox2 were initialized by the Box( ) constructor when they were created.
- Since the constructor gives all boxes the same dimensions, 10 by 10 by 10, both mybox1 and mybox2 will have the same volume.
- The println( ) statement inside Box( ) is for the sake of illustration only.
- Most constructors will not display anything.
- They will simply initialize an object.

Parameterized Constructors:

- The Box( ) constructor in the preceding example does initialize a Box object.
- But, it is not very useful—all boxes have the same dimensions.
- What is needed is a way to construct Box objects of various dimensions.
- The easy solution is to **add parameters to the constructor.**
- The following version of Box defines a parameterized constructor that sets the dimensions of a box as specified by those parameters.

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
class Box {
  double width;
  double height;
  double depth;

  // This is the constructor for Box.
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo7 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box(3, 6, 9);

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

OUTPUT:

```
Volume is 3000.0
Volume is 162.0
```

Note:

- Each object is initialized as specified in the parameters to its constructor.
- For example, in the following line,

```
Box mybox1 = new Box(10, 20, 15);
```

- Here, the values 10, 20, and 15 are passed to the Box( ) constructor when new creates the object.
- Thus, mybox1's copy of width, height, and depth will contain the values 10, 20, and 15, respectively.

The this Keyword:

- The *this* keyword in Java is a **reference to the current object (the object whose method or constructor is being called)**
- *this* can be used inside any method to refer to the current object.
  - ie, *this* always a reference to the object on which the method was invoked.
  - We can use *this* anywhere a reference to an object of the current class' type is permitted.

- Consider the program of page 111.
- The following is an alternative version of Box( ) which operates exactly like the earlier version (Program of page 111).
- Inside Box( ), this will always refer to the invoking object.

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
class Box {
  double width;
  double height;
  double depth;

// A redundant use of this.
Box(double w, double h, double d) {
  this.width = w;
  this.height = h;
  this.depth = d;
}

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo7 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box(3, 6, 9);

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

OUTPUT:

```
Volume is 3000.0
Volume is 162.0
```

Using *this* keyword to resolve the ambiguity between formal parameters and instance variables:

- Consider the following example.

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
class Box {
  double width;
  double height;
  double depth;

Box(double width, double height, double depth) {
  this.width = width;
  this.height = height;
  this.depth = depth; }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo7 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box(3, 6, 9);

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

- Here, the class has instance variables - **width, height and depth.**
- The constructor has formal parameters - **width, height and depth.**

- Now the formal parameters and instance variables have exactly the same names.
- Here;

  this.width = width;
- This means:
  - this.width → instance variable of the current object.
  - width → formal parameter of the constructor.
- this.width = width;

  ↓ ↓

  instance variable parameter
- So, this is used to explicitly tell Java that we mean the instance variable of the current object, especially when its name is the same as a parameter.
- This resolves the ambiguity between formal parameters and instance variables.

Using *this* keyword for instance variable hiding:

- Consider the following example.

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
class Box {
  double width;
  double height;
  double depth;

Box(double width, double height, double depth) {
  this.width = width;
  this.height = height;
  this.depth = depth; }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo7 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box(3, 6, 9);

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

- Here, the class has instance variables - **width, height and depth.**
- The constructor has formal parameters - **width, height and depth.**

- Now the formal parameters and instance variables have exactly the same names.
- Now suppose the constructor uses the same names.

```
// Use this to resolve name-space collisions.
Box(double width, double height, double depth) {
  this.width = width;
  this.height = height;
  this.depth = depth;
}
```

- Instance variable ↓ this.width Formal parameter ↓ width
- But inside the constructor, when Java sees: width, it gives priority to the nearest/local variable, which is the parameter.
- Therefore: this.width = width; is interpreted as:

  parameter width = parameter width
- This is called variable hiding
- The parameter width hides the instance variable width within the constructor.

PROCEDURAL V/S OBJECT-ORIENTED PARADIGM

Procedural Programming Language vs Object-Oriented Programming (OOP)

Aspect	Procedural Programming Language	Object-Oriented Programming (OOP) Language
Definition	A programming paradigm that organizes code into procedures (functions).	A programming paradigm that organizes code into objects containing data and methods.
Approach	Top-down approach	Bottom-up approach
Basic Unit	Functions/Procedures	Objects and Classes
Focus	Focuses on functions and the sequence of actions	Focuses on data (objects) and their interactions
Data Security	Data is generally shared globally and is less secure	Data is hidden and accessed through methods, providing better security
Data & Functions	Data and functions are separate	Data and functions are combined into objects
Reusability	Limited code reusability through functions	High code reusability through classes and objects
Modularity	Programs are divided into functions	Programs are divided into classes and objects
Maintenance	Difficult to maintain large programs	Easier to maintain and extend large programs
Examples of Languages	C, Pascal, FORTRAN	Java, C++, Python, C#, Ruby

DATA ABSTRACTION:

- An essential element of object-oriented programming is abstraction.
- Abstraction means **displaying only essential information and hiding the inner details.**
- It **consider only those aspects of the problem that are relevant to a given purpose suppresses all aspects of the problem that are not relevant.**
- A powerful way to manage abstraction is through the use of hierarchical classifications.
- Abstraction is supported in two different ways in an object-oriented designs (OODs).
  1. **Feature abstraction:**
     - A class hierarchy can be viewed as different levels (hierarchy) of abstraction,
     - Class hierarchy- class can have subclasses.(Inheritance).

  2. Data abstraction:

  Each object hides the way in which it stores its internal information from other objects.

Abstraction - Real world example:

- Consider a man driving a car.
- The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car.
- But he may not know about how on pressing accelerator the speed is actually increasing.
- He may not know about the inner mechanism of the car or the implementation of accelerator, brakes etc in the car.
- This is what abstraction is.
- This abstraction allows people to use a car to drive to the grocery store without being overwhelmed by the complexity of the parts that form the car.
- They can ignore the details of how the engine, transmission, and braking systems work.
- Instead, they are free to utilize the object as a whole.

Advantages - Data Abstraction:

- It reduces coupling among various objects
- Objects do not directly access any data belonging to each other.
- It leads to a reduction of the overall complexity of a design.
- It helps in easy maintenance and code reuse.

THE THREE OOP PRINCIPLES

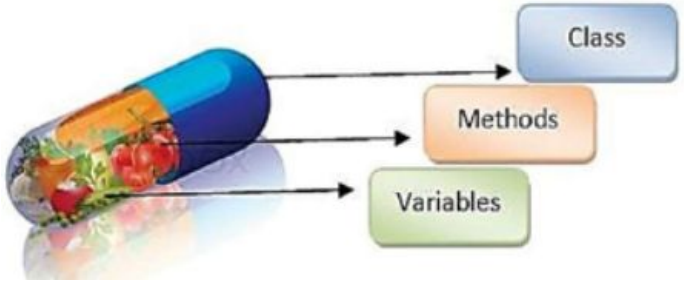
-Object-oriented programming (OOP) is at the core of Java.

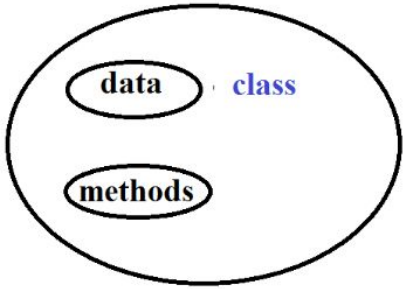
-In fact, all Java programs are to atleast some extent object-oriented.

- 3 principles;

- **Encapsulation**
- **Inheritance**
- **Polymorphism.**

ENCAPSULATION





- The wrapping up of data (variables) and functions (methods) into a single unit(class) is called Encapsulation.
- Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.
- Encapsulation can be thought of as **a protective wrapper that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper**.
- Access to the code and data inside the wrapper is tightly controlled through a well-defined interface.

- Each method or variable in a class may be marked private or public.
- The public interface of a class represents everything that external users of the class need to know, or may know.
- The private methods and data can only be accessed by code that is a member of the class.
- Since the private members of a class may only be accessed by other parts of our program through the class' public methods, we can ensure that no improper actions take place.
- This means that the public interface should be carefully designed not to expose too much of the inner workings of a class.

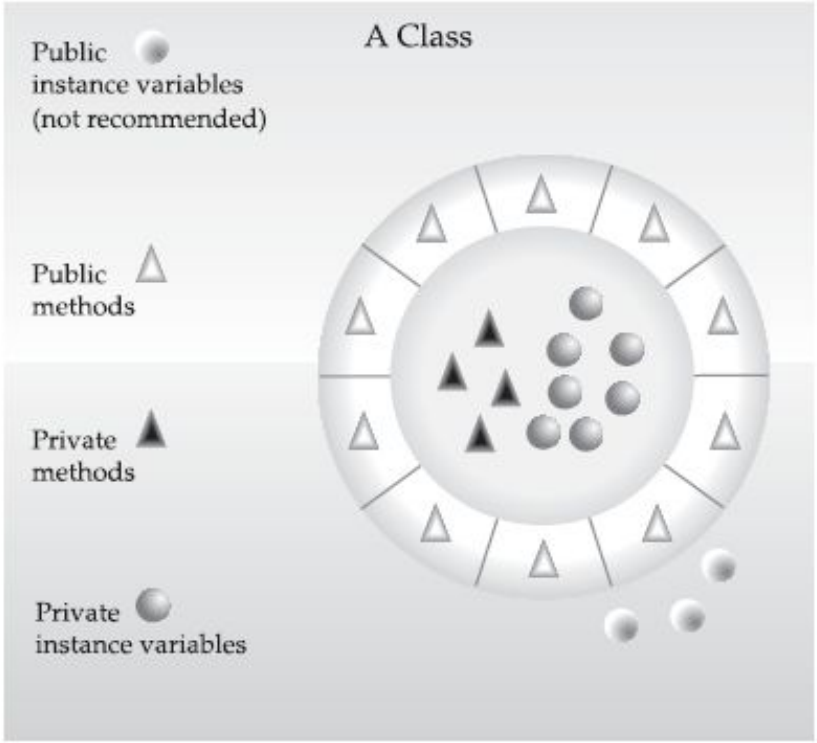


Fig: Encapsulation: public methods can be used to protect private data.

Advantages -Encapsulation:

- **Protection from unauthorized data access:**
  - Protect data from accidental corruption and concurrent access(simultaneous access) problems.
- **Data hiding:**
  - Helps to hide the internal structure data of an object.
  - Provides abstraction, easier maintenance and bug correction.
- **Weak coupling:**
  - Since objects do not directly change each others internal data, they are weakly coupled.
  - This enhances understandability of the design.

INHERITANCE

- Inheritance can be defined as the **capability of a class to derive properties and characteristics from another class.**
- It is the process by which **one object acquires the properties of another object.**
- It promotes code reusability and establishes an "is-a" relationship between classes.

Inheritance Terminology in Java:

- Base Class (Parent Class / Superclass)
- Derived Class (Child Class / Subclass)

Base Class (Parent Class / Superclass)

- A Base Class is the class whose properties (fields) and methods are inherited by another class.
- Also called Parent Class or Superclass.
- It contains common attributes and behaviors that can be shared by other classes.

Derived Class (Child Class / Subclass)

- A Derived Class is the class that inherits the members of the base class.
- Also called Child Class or Subclass.
- It can use the inherited members and also define its own members.

Eg: Doctor is a superclass. Surgeon and Neurologist are its subclasses

Subclasses inherit properties from Super class.

- A base class can be **specialized** into derived classes.
  - Eg: Employee - Teacher, Manager, Banker
- A base class is a **generalization** of its derived classes.
- Base class contains properties (i.e., data and methods) that are common to all its derived classes.
- Derived class inherit the properties of base class.
- Derived class can have its own special properties also.
  - Eg: Teacher class inherits properties from Employee class.
    - Teacher class also has its own special properties.

Base class - Derived Class:

- Each derived class can be considered as a specialization of its base class.
- Because it modifies or extends the basic properties of the base class in certain ways.
- Therefore, the inheritance relationship can be viewed as a **generalization-specialization relationship.**

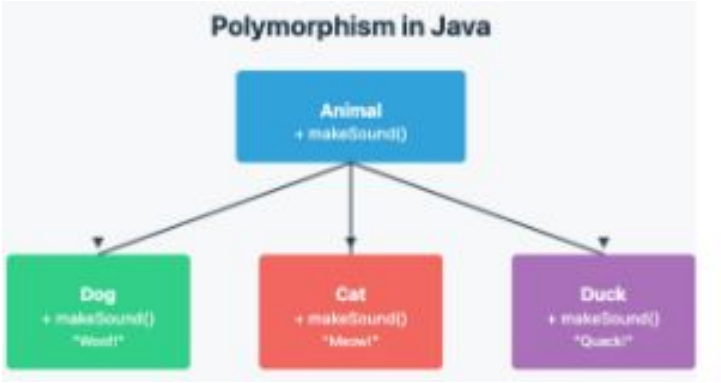
POLYMORPHISM:

- Polymorphism literally means poly (**many**) morphism (**forms**).
- Real world example:Diamond, graphite, and coal are called polymorphic forms of carbon.
- Polymorphism is a feature that **allows one interface to be used for a general class of actions.**
- The concept of polymorphism is often expressed by the phrase; **“one interface, multiple methods”**
- This means that, it is possible to design a generic interface to a group of related activities.
- This reduce complexity by allowing the same interface to be used to specify a general class of action.
- It is the compiler’s job to select the specific action (that is, method) as it applies to each situation.

Eg 1:

- Consider a stack.
- You might have a program that requires three types of stacks.
  - One stack is used for integer values
  - One for floating-point values
  - One for characters.
- The algorithm that implements each stack is the same, eventhough the data being stored differs.
- In a non–object-oriented language, you would be required to create three different sets of stack routines, with each set using different names.
- In Java, you can specify a general set of stack routines that all share the same names.
  - This is because of polymorphism.

Eg 2:



- There is a superclass: Animal
  - makeSound()
- There are 3 sub classes
  - Dog->makeSound() -> “Bark”
  - Cat->makeSound() -> “Meow”
  - Duck>makeSound() -> “Quack”
- All three classes have the same method makeSound(), but each provides it’s own implementation.
- Here, the same method call animal.makeSound() can produce different results depending on the actual object. ===> ***POLYMORPHISM***

ARRAYS

- An array is a group of like-typed variables that are referred to by a common name.
- Arrays of any type can be created and may have one or more dimensions.
- A specific element in an array is accessed by its index.
- Arrays offer a convenient means of grouping related information.

One-Dimensional Arrays

- A one-dimensional array is, essentially, a list of like-typed variables.
- To create an array, we must create an array variable of the desired type.
- The general form of a one-dimensional array declaration is

```
type array-name[ ];
```

  - type declares the base type of the array.
  - The base type determines the data type of each element that comprises the array.

Eg:

```
int month_days[ ];
```

- This statement declares an array named month_days of type int.
- This declaration establishes that month_days is an array variable.
- But, no array actually exists.
- To link month_days with an actual, physical array of integers, we must allocate one using new and assign it to month_days.
- **new** is a special operator that allocates memory.
- The general form of new as it applies to one-dimensional arrays appears as follows:

array-name = new type[size];

- type specifies the type of data being allocated, size specifies the number of elements in the array, and array-name is the array variable that is linked to the array.
- That is, to use new to allocate an array, we must specify the type and number of elements to allocate.
- The elements in the array allocated by new will automatically be initialized to zero.

- Eg:

```
month_days = new int[12]; // allocates a 12-element array of integers and links them to month_days.
```

- After this statement executes, month_days will refer to an array of 12 integers.
- Further, all elements in the array will be initialized to zero.

Summary:

- Obtaining an array is a two-step process.
- First, you must declare a variable of the desired array type.
- Second, you must allocate the memory that will hold the array, using new, and assign it to the array variable.
- Thus, **in Java all arrays are dynamically allocated.**
- Once you have allocated an array, you can access a specific element in the array by specifying its index within square brackets.
- All array indexes start at zero.

- Eg:

```
month_days[1] = 28;    //assigns the value 28 to the second element of month_days.

System.out.println(month_days[3]);     // displays the value stored at index 3.
```

USING COMMAND LINE ARGUMENTS

- Sometimes you will want **to pass information into a program when you run it.**
- This is accomplished by passing command-line arguments to main( ).
- A command-line argument i**s the information that directly follows the program's name on the command line when it is executed**.
- It is quite easy to access the command-line arguments inside a Java program - they are **stored as strings in a String array passed to the args parameter of main( )**.
- The first command-line argument is stored at args[0], the second at args[1], and so on.

Eg: The following program displays all of the command-line arguments that it is called with:

```
// Display all command-line arguments.
class CommandLine {
  public static void main(String args[]) {
    for(int i=0; i<args.length; i++)
      System.out.println("args[" + i + "]: " +
                         args[i]);
  }
}
```

Try executing this program, as shown here:

```
java CommandLine this is a test 100 -1
```

When you do, you will see the following output:

```
args[0]: this
args[1]: is
args[2]: a
args[3]: test
args[4]: 100
args[5]: -1
```